AMRITA VISHWA VIDYAPEETHAM CHENNAI CAMPUS

Amrita School of Computing

Coding Practice

Activity Sheets

(For the Batch 2025-2029)

Name:Ansika Pandey
Roll No.: CH.SC.U4CSE25269
Department: CSE "C"

Activity Sheet 05 **Start Date: 30/05/2026 End Date: 02/06/2026.**

Topic: Searching [Easy + Medium Coding Questions]

Instructions

The Mock Test syllabus for CSE CT | CYS | AIE | RAI program is enclosed in the attached document for your reference.

Materials for the Mock Test [Must-DO Coding Questions] is provided in a link.

It is mandatory to upload the completed Activity sheets before the deadline in the MS-Form link shared by your class Advisor.

Activity Sheet: 01 - Coding Questions from Searching – Part 1 [Easy + Medium Questions]

Each Activity Sheets contains total of 20 Questions.

Platform: Any Online Compiler of your choice – GDB Compiler

Language: C | C++ | JAVA | PYTHON

The Activity Sheet will have the coding Questions and the Test Case. Once executed in Online Compiler, take screen shot of the code and the output and paste it in the space provided in the activity sheet.

The Final Activity Sheet should be in PDF Format and the file name should be your roll number.

Course Instructors: CARE TEAM
Dr.R.Annamalai Assistant Professor (Sl.Grade) |CSE-AIE
Dr.J.Umamageswaran Assistant Professor (Sr.Grade) | CSE-CT
Ms.D. Sasikala Assistant Professor | CSE-AIE
Mr.NirmalRaj Assistant Professor | CSE-CYS

Activity Sheet 05 **Start Date: 30/05/2026 End Date: 02/06/2026.**

Topic: Searching [Easy + Medium Coding Questions]

1. Given an arr[] of elements of size n, return the largest element given in the array.

Examples:
Input: arr[] = [10, 20, 4]
Output: 20
Explanation: Among 10, 20 and 4, 20 is the largest.

Input: arr[] = [20, 10, 20, 4, 100]
Output: 100

Code:

```
import java.util.*;

public class Main {

    public static int largestElement(int[] arr) {
        int max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) {
                max = arr[i];
            }
        }
        return max;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        System.out.println(largestElement(arr));
        sc.close();
    }
}
```

Output:

```
3
10 20 4
20
5
20 10 20 4 100
100
```

2. You are given an array arr[] of size n - 1 that contains distinct integers in the range from 1 to n (inclusive). This array represents a permutation of the integers from 1 to n with one element missing. Your task is to identify and return the missing element.

Constraints:
$1 \leq arr.size() \leq 106$
$1 \leq arr[i] \leq arr.size() + 1$

Input: arr[] = [1, 2, 3, 5]
Output: 4
Explanation: All the numbers from 1 to 5 are present except 4.

Input: arr[] = [1]
Output: 2
Explanation: Only 1 is present so the missing element is 2.

Input: arr[] = [8, 2, 4, 5, 3, 7, 1]
Output: 6
Explanation: All the numbers from 1 to 8 are present except 6.

Code:

```
import java.util.*;

public class Main {
    public static int findMissing(int[] arr) {
        int n = arr.length + 1;
        long totalSum = (long) n * (n + 1) / 2;
        long actualSum = 0;
        for (int num : arr) {
            actualSum += num;
        }
        return (int)(totalSum - actualSum);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        System.out.println(findMissing(arr));
        sc.close();
    }
}
```

Output

```
1
1
2
 4
 1 2 3 5
 4
7
8 2 4 5 3 7 1
6
```

3. Given an array arr[] containing only non-negative integers, your task is to find a continuous subarray (a contiguous sequence of elements) whose sum equals a specified value target. You need to return the 1-based indices of the leftmost and rightmost elements of this subarray. You need to find the first subarray whose sum is equal to the target.

Note: If no such array is possible then, return [-1].
Constraints:
$1 \leq \text{arr.size()} \leq 106$
$0 \leq \text{arr[i]} \leq 103$
$0 \leq \text{target} \leq 109$

Input: arr[] = [1, 2, 3, 7, 5], target = 12
Output: [2, 4]
Explanation: The sum of elements from 2nd to 4th position is 12.

Input: arr[] = [5, 3, 4], target = 2
Output: [-1]
Explanation: There is no subarray with sum 2.

Code:

```
import java.util.*;

public class Main {
    public static int[] findSubarray(int[] arr, long target) {
        int left = 0;
        long sum = 0;
        for (int right = 0; right < arr.length; right++) {
            sum += arr[right];
            while (sum > target && left <= right) {
                sum -= arr[left];
                left++;
            }
            if (sum == target) {
                return new int[]{left + 1, right + 1}; // 1-based indexing
            }
        }
        return new int[]{-1};
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        long target = sc.nextLong();
        int[] result = findSubarray(arr, target);
        for (int x : result) {
            System.out.print(x + " ");
        }
        sc.close();
    }
}
```

Output

```
-
5
1 2 3 7 5
12
2 4
-
3
5 3 4
2
-1
```

4. Given an array of positive integers arr[], return the second largest element from the array. If the second largest element doesn't exist then return -1.

Note: The second largest element should not be equal to the largest element.

Code:

```
import java.util.*;

public class Main {
    public static int secondLargest(int[] arr) {
        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;
        for (int num : arr) {
            if (num > largest) {
                secondLargest = largest;
                largest = num;
            }
            else if (num < largest && num > secondLargest) {
                secondLargest = num;
            }
        }
        return (secondLargest == Integer.MIN_VALUE) ? -1 : secondLargest;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        System.out.println(secondLargest(arr));
        sc.close();
    }
}
```

Output

```
5
12 35 1 10 34 1
34
3
10 10 10
-1
```

5. Given an array arr[]. Find the majority element in the array. If no majority element exists, return -1.

Constraints:
$1 \leq arr.size() \leq 105$
$1 \leq arr[i] \leq 105$

Note: A majority element in an array is an element that appears strictly more than arr.size()/2 times in the array.
Input: arr[] = [1, 1, 2, 1, 3, 5, 1]
Output: 1
Explanation: Since, 1 is present more than 7/2 times, so it is the majority element.

Input: arr[] = [2, 13]
Output: -1
Explanation: Since, no element is present more than 2/2 times, so there is no majority element.

Code:

```
import java.util.*;

public class Main {
    public static int majorityElement(int[] arr) {
        int candidate = -1;
        int count = 0;
        for (int num : arr) {
            if (count == 0) {
                candidate = num;
                count = 1;
            } else if (num == candidate) {
                count++;
            } else {
                count--;
            }
        }
        int freq = 0;
        for (int num : arr) {
            if (num == candidate) {
                freq++;
            }
        }
        if (freq > arr.length / 2) {
            return candidate;
        }
        return -1;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        System.out.println(majorityElement(arr));
        sc.close();
    }
}
```

Output

```
7
1 1 2 1 3 5 1
1

2
2 13
-1
```

6. Given an integer array arr[] and an integer k, your task is to find and return the kth smallest element in the given array.

Constraints:
$1 \leq arr.size() \leq 105$
$1 \leq arr[i] \leq 105$
$1 \leq k \leq arr.size()$

Note: The kth smallest element is determined based on the sorted order of the array.
Input: arr[] = [10, 5, 4, 3, 48, 6, 2, 33, 53, 10], k = 4
Output: 5
Explanation: 4th smallest element in the given array is 5.

Input: arr[] = [7, 10, 4, 3, 20, 15], k = 3
Output: 7
Explanation: 3rd smallest element in the given array is 7.

Code:

```
import java.util.*;

public class Main {
    public static int kthSmallest(int[] arr, int k) {
        Arrays.sort(arr);
        return arr[k - 1];
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int k = sc.nextInt();
        System.out.println(kthSmallest(arr, k));
        sc.close();
    }
}
```

Output

```
10
10 5 4 3 48 6 2 33 53 10
4
5

 6
 7 10 4 3 20 15
 3
 7
```

7. Given a sorted array and a number key, return whether the key is present in the array or not.

Constraints
1 <= T <= 100
1 <= n <= 104
-106 <= Ai <= 106
-106 <= key <= 106

Examples
Array: [1, 2, 3, 3, 3, 4, 4, 5]
Number: 2
Answer: true

Array: [1, 2, 3, 3, 3, 4, 4, 5]
Number: 6
Answer: false

Input Format
First-line contains an integer ‘T’ denoting the number of test cases.
For each test case the input has two lines:
Two space-separated integers ‘n’ and 'key' denoting the length of the array and the number to be searched respectively. n space-separated integers denoting the elements of the array.

Output Format
T lines each contain true or false denoting the answer for each test case.
Sample Input
2
8 2
1 2 3 3 3 4 4 5
8 6
1 2 3 3 3 4 4 5

Expected Output
true
false

Code:

```
import java.util.*;

public class Main {
    public static boolean binarySearch(int[] arr, int key) {
        int low = 0;
        int high = arr.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == key) {
                return true;
            } else if (arr[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return false;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        while (T-- > 0) {
            int n = sc.nextInt();
            int key = sc.nextInt();
            int[] arr = new int[n];
            for (int i = 0; i < n; i++) {
                arr[i] = sc.nextInt();
            }
            System.out.println(binarySearch(arr, key));
        }
        sc.close();
    }
}
```

Output

```
2
8 2
1 2 3 3 3 4 4 5
true
8 6
1 2 3 3 3 4 4 5
false
```

8. Given a sorted array and a number key, find the index of the first and last occurrence of the key in the array.

If the key is not present, return [-1, -1].

Constraints
1 <= T <= 100
1 <= n <= 104
-106 <= Ai <= 106
-106 <= key <= 106

Examples
Array: [1, 2, 3, 3, 3, 4, 4, 5]
Number: 3
Answer: [2, 4]

Array: [1, 2, 3, 3, 3, 4, 4, 5]
Number: 5
Answer: [7, 7]

Array: [1, 2, 3, 3, 3, 4, 4, 5]
Number: 6
Answer: [-1, -1]

Testing
Input Format
First-line contains an integer ‘T’ denoting the number of test cases.
For each test case the input has two lines:
Two space-separated integers ‘n’ and 'key' denoting the length of the array and the number to be searched respectively.
n space-separated integers denoting the elements of the array.

Output Format
T lines each contain two integers denoting the indices of the first and last occurrence of the key in the array for each test case.

Sample Input
2
8 3
1 2 3 3 3 4 4 5
8 6
1 2 3 3 3 4 4 5
Expected Output
2 4
-1 -1

Code:

```
import java.util.*;

public class Main {
    static int firstOccurrence(int[] arr, int key) {
        int low = 0, high = arr.length - 1;
        int result = -1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == key) {
                result = mid;
                high = mid - 1;
            } else if (arr[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return result;
    }
    static int lastOccurrence(int[] arr, int key) {
        int low = 0, high = arr.length - 1;
        int result = -1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == key) {
                result = mid;
                low = mid + 1;
            } else if (arr[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return result;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        while (T-- > 0) {
            int n = sc.nextInt();
            int key = sc.nextInt();
```

```
            int[] arr = new int[n];
            for (int i = 0; i < n; i++) {
                arr[i] = sc.nextInt();
            }
            int first = firstOccurrence(arr, key);
            int last = lastOccurrence(arr, key);
            System.out.println(first + " " + last);
        }
        sc.close();
    }
}
```

Output

```
2
8 3
1 2 3 3 3 4 4 5
2 4
8 6
1 2 3 3 3 4 4 5
-1 -1
```

9. Given a sorted array of integers, find the number of negative numbers.

Constraints
1 <= T <= 100
1 <= n <= 104
-105 <= Ai <= 105

Examples
Array: [-5, -3, -2, 3, 4, 6, 7, 8]
Answer: 3

Array: [0, 1, 2, 3, 4, 6, 7, 8]
Answer: 0

Testing
Input Format
The first line contains 'T' denoting the no. of test cases.
T lines each contain a number 'n' denoting the number of elements, followed by n space-separated numbers denoting the array elements.

Output Format
T lines each contain an integer denoting the number of negative numbers for that test case.

Sample Input
2
8 -5 -3 -2 3 4 6 7 8
8 0 1 2 3 4 6 7 8
Expected Output
3
0

Code:

```
import java.util.*;

public class Main {
    public static int countNegatives(int[] arr) {
        int low = 0;
        int high = arr.length - 1;
        int firstNonNegative = arr.length;
        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (arr[mid] >= 0) {
                firstNonNegative = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        return firstNonNegative;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        while (T-- > 0) {
            int n = sc.nextInt();
            int[] arr = new int[n];
            for (int i = 0; i < n; i++) {
                arr[i] = sc.nextInt();
            }
            System.out.println(countNegatives(arr));
        }
        sc.close();
    }
}
```

Output

```
2
8
-5 -3 -2 3 4 6 7 8
3
8
0 1 2 3 4 6 7 8
0
```

10. Given a sorted array and a number key, find the smallest array element which is greater than the key.

If the key is greater than or equal to the largest element then return the key itself.

Constraints
1 <= T <= 100
1 <= n <= 104
-106 <= Ai <= 106
-106 <= key <= 106

Examples
Array: [1, 2, 3, 3, 4, 4, 8, 10]
Number: 4
Answer: 8

Array: [1, 2, 3, 3, 3, 4, 4, 5]
Number: 5
Answer: 5 (Largest Element)

Array: [1, 2, 3, 3, 3, 4, 4, 5]
Number: -5
Answer: 1

Testing
Input Format
First-line contains an integer ‘T’ denoting the number of test cases.
For each test case the input has two lines:
Two space-separated integers ‘n’ and 'key' denoting the length of the array and the number to be searched respectively.
n space-separated integers denoting the elements of the array.
Output Format
T lines each contain the next greater element for each test case.

Sample Input
2
8 4
1 2 3 3 4 4 8 10
8 5
1 2 3 3 3 4 4 5
Expected Output
8
5

Code:

```
import java.util.*;

public class Main {
    public static int nextGreaterElement(int[] arr, int key) {
        int low = 0;
        int high = arr.length - 1;
        int ans = key;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] > key) {
                ans = arr[mid];
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        return ans;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        while (T-- > 0) {
            int n = sc.nextInt();
            int key = sc.nextInt();
            int[] arr = new int[n];
            for (int i = 0; i < n; i++) {
                arr[i] = sc.nextInt();
            }
            System.out.println(nextGreaterElement(arr, key));
        }
        sc.close();
    }
}
```

Output

```
2
8 4
1 2 3 3 4 4 8 10
8
8 5
1 2 3 3 3 4 4 5
5
```

11. Given a sorted array containing distinct integers and a number 'key', find the index of the key in the array. If the key is not present, return the index at which it would be inserted considering that we need to maintain the sort order.

Constraints
1 <= T <= 100
1 <= n <= 104
-106 <= Ai <= 106
-106 <= key <= 106

Examples
Array: [1, 2, 3, 4, 5]
Number: 3
Answer: 2

Array: [1, 2, 3, 5]
Number: 4
Answer: 3

Array: [1, 2, 3, 4, 5]
Number: -100
Answer: 0

Array: [1, 2, 3, 4, 5]
Number: 6
Answer: 5

Testing
Input Format
First-line contains an integer ‘T’ denoting the number of test cases.
For each test case the input has two lines:
Two space-separated integers ‘n’ and 'key' denoting the length of the array and the number to be searched respectively.
n space-separated integers denoting the elements of the array.
Output Format
T lines each contain the index of the first occurrence of the key in the array or the index to insert the number (if not present) for each test case.

Code:

```
import java.util.*;

public class Main {
    public static int searchInsert(int[] arr, int key) {
        int low = 0;
        int high = arr.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == key) {
                return mid;
            } else if (arr[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return low;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        while (T-- > 0) {
            int n = sc.nextInt();
            int key = sc.nextInt();
            int[] arr = new int[n];
            for (int i = 0; i < n; i++) {
                arr[i] = sc.nextInt();
            }
            System.out.println(searchInsert(arr, key));
        }
        sc.close();
    }
}
```

Output

```
4
5 3
1 2 3 4 5
2
4 4
1 2 3 5
3
5 -100
1 2 3 4 5
0
5 6
1 2 3 4 5
5
```

12. You are given a list of unique integers which are sorted but rotated at some pivot. You are also given a target value and you have to find its index in the list. If it is not present in the list, return -1.

Constraints
1 <= T <= 100
1 <= n <= 104
1 <= array elements <= 106
1 <= target <= 106

Example:
List: [4, 5, 6, 7, 1, 2, 3]
Target value: 6
Resultant index: 2

Testing
Input Format
The first line contains 'T', denoting the number of test cases.

Each test contains 3 lines:

a number 'n', denoting the number of elements.
n space-separated numbers denoting the array elements.
the target value.
Output Format
T lines, each containing a number denoting the index of the target value. -1 if the target value is not present.

Sample Input
4
7
4 5 6 7 0 1 2
4
4
3 4 1 2
5
5
5 1 2 3 4
2
4
5 6 3 4
4
Expected Output
0
-1
2
3

Activity Sheet 05 **Start Date: 30/05/2026 End Date: 02/06/2026.**

Topic: Searching [Easy + Medium Coding Questions]

Code:

```
import java.util.*;

public class Main {
    public static int search(int[] arr, int target) {
        int low = 0;
        int high = arr.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == target) {
                return mid;
            }
            if (arr[low] <= arr[mid]) {
                if (target >= arr[low] && target < arr[mid]) {
                    high = mid - 1;
                } else {
                    low = mid + 1;
                }
            }
            else {
                if (target > arr[mid] && target <= arr[high]) {
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
        }
        return -1;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        while (T-- > 0) {
            int n = sc.nextInt();
            int[] arr = new int[n];
            for (int i = 0; i < n; i++) {
                arr[i] = sc.nextInt();
            }
            int target = sc.nextInt();
            System.out.println(search(arr, target));
        }
```

```
        sc.close();
    }
}
```

Output

```
4
7
4 5 6 7 0 1 2
4
0
4
3 4 1 2
5
-1
5
5 1 2 3 4
2
2
4
5 6 3 4
4
3
```

13. Given a sorted list of numbers in which all elements appear twice except one element that appears only once, find the number that appears only once.

Constraints
1 <= T <= 100
1 <= Size of array <= 20000
1 <= Elements in array <= 106

Example:
1, 1, 2, 3, 3, 4, 4
Here, '2' appears once and all other elements appear twice.

findNonRepeatingElement([1, 1, 2, 3, 3, 4, 4]) => 2
Expected Time Complexity: O(log n)

Testing
Input Format
The first line contains 'T', denoting the no. of test cases.

Each test case consists of two lines. The first contains a number 'n' denoting the number of elements. The second line has 'n' space-separated numbers denoting the elements.

Output Format
For each test case, a line containing the non-repeating number.

Sample Input
3
1
3
3
1 2 2
5
3 3 4 4 5
Expected Output
3
1
5

Code:

```
import java.util.*;

public class Main {
    public static int findNonRepeatingElement(int[] arr) {
        int low = 0;
        int high = arr.length - 1;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (mid % 2 == 1) {
                mid--;
            }
            if (arr[mid] == arr[mid + 1]) {
                low = mid + 2;
            } else {
                high = mid;
            }
        }
        return arr[low];
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        while (T-- > 0) {
            int n = sc.nextInt();
            int[] arr = new int[n];
            for (int i = 0; i < n; i++) {
                arr[i] = sc.nextInt();
            }
            System.out.println(findNonRepeatingElement(arr));
        }
        sc.close();
    }
}
```

Output

```
3
1
3
3
3
1 2 2
1
5
3 3 4 4 5
5
```

14. Given a matrix, check if the matrix contains a given key.

The matrix has the following properties:

Integer in each row is arranged in non-decreasing order from left to right.
The first integer in every row is greater than the last integer of the previous row.

Constraints
1 <= T <= 10
1 <= n,m <= 100
0 <= key <= 105
0 <= matrix[i][j] <= 105

Examples
Matrix:
1 2 3
4 5 6
7 8 9
Key: 6
Answer: true
Matrix:
1 2 3
4 5 6
7 8 9
10 11 12
Key: 15
Answer: false
Testing
Sample Input
The first line contains 'T' denoting the no. of test cases.

For each test case:

The first line contains the numbers 'n' and 'm' denoting the number of rows and columns respectively followed by the key.
The next ‘n’ lines contain ‘m’ space-separated integers denoting elements of a 2-d matrix.
Expected Output
T lines each contain true/false denoting the answer.

Sample Input
2
3 3 6
1 2 3
4 5 6
7 8 9
4 3 15
1 2 3
4 5 6
7 8 9
10 11 12
Expected Output
true
false

Code:

```
import java.util.*;

public class Main {
    public static boolean searchMatrix(int[][] matrix, int n, int m, int key) {
        int low = 0;
        int high = n * m - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            int row = mid / m;
            int col = mid % m;
            if (matrix[row][col] == key) {
                return true;
            } else if (matrix[row][col] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return false;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        while (T-- > 0) {
            int n = sc.nextInt();
            int m = sc.nextInt();
            int key = sc.nextInt();
            int[][] matrix = new int[n][m];
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < m; j++) {
                    matrix[i][j] = sc.nextInt();
                }
            }
            System.out.println(searchMatrix(matrix, n, m, key));
        }
        sc.close();
    }
}
```

Output

```
2
3 3 6
1 2 3
4 5 6
7 8 9
true
4 3 15
1 2 3
4 5 6
7 8 9
10 11 12
false
```

15. A peak element is an element strictly greater than its neighbors. Given an integer array arr[], find any peak element's index using binary search. If the array contains multiple peaks, return any one.

Constraints:

$1 \leq n \leq 10^3$ | $arr[-1] = arr[n] = -\infty$ | $arr[i] \neq arr[i+1]$

Example:

Input

arr=[1,2,3,1]

Output

2

Input

arr=[1,2,1,3,5,6,4]

Output

5 (or 1)

Input

arr=[1]

Output

0

Code:

```
public class PeakElement {

    public static int findPeakElement(int[] arr) {
        int low = 0;
        int high = arr.length - 1;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] < arr[mid + 1]) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        return low;
    }
    public static void main(String[] args) {
        int[] arr1 = {1, 2, 3, 1};
        System.out.println(findPeakElement(arr1));
        int[] arr2 = {1, 2, 1, 3, 5, 6, 4};
        System.out.println(findPeakElement(arr2));
        int[] arr3 = {1};
        System.out.println(findPeakElement(arr3));
    }
}
```

Output

```
2
5
0
```

16. Given an array arr[] and a target, determine if there exist three elements that sum to the target. Return "YES" or "NO".
Constraints: 3 <= n <= 10^3 | -10^9 <= arr[i] <= 10^9

Input
arr=[1,4,45,6,10,8], target=22
Output
YES (4+10+8)

Input
arr=[1,2,4,3,6], target=10
Output
YES (1+3+6)

Input
arr=[1,2,3,4,5], target=100
Output
NO

Code:

```
import java.util.Arrays;

public class TripletSum {
    public static String findTriplet(int[] arr, int target) {
        Arrays.sort(arr);
        int n = arr.length;
        for (int i = 0; i < n - 2; i++) {
            int left = i + 1;
            int right = n - 1;
            while (left < right) {
                long sum = (long) arr[i] + arr[left] + arr[right];
                if (sum == target) {
                    return "YES";
                } else if (sum < target) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return "NO";
    }
    public static void main(String[] args) {
        int[] arr1 = {1, 4, 45, 6, 10, 8};
        System.out.println(findTriplet(arr1, 22));
        int[] arr2 = {1, 2, 4, 3, 6};
        System.out.println(findTriplet(arr2, 10));
        int[] arr3 = {1, 2, 3, 4, 5};
        System.out.println(findTriplet(arr3, 100));
    }
}
```

Output

```
YES
YES
NO
```

17. Given an m×n matrix where each row is sorted left-to-right and each column is sorted top-to-bottom, return true if the target exists, false otherwise.

Constraints:

$1 \le m, n \le 300$ | $-10^9 \le matrix[i][j], target \le 10^9$

Input:
matrix=[[1,4,7],[2,5,8],[3,6,9]], target=5
Output:
true

Input:
matrix=[[1,4,7],[2,5,8],[3,6,9]], target=10
Output:
false

Code:

```
public class SearchSortedMatrix {
    public static boolean searchMatrix(int[][] matrix, int target) {
        int m = matrix.length;
        int n = matrix[0].length;
        int row = 0;
        int col = n - 1;
        while (row < m && col >= 0) {
            if (matrix[row][col] == target) {
                return true;
            } else if (matrix[row][col] > target) {
                col--;
            } else {
                row++;
            }
        }
        return false;
    }
    public static void main(String[] args) {
        int[][] matrix1 = {
            {1, 4, 7},
            {2, 5, 8},
            {3, 6, 9}
        };
        System.out.println(searchMatrix(matrix1, 5));
        System.out.println(searchMatrix(matrix1, 10));
    }
}
```

Output

```
true
false
```

18. Given a sorted array arr[] and an integer x, find the number of occurrences of x in the array using binary search.

Constraints:
$1 \leq n \leq 10^5$ | $1 \leq arr[i] \leq 10^5$

Input:
arr=[1,1,2,2,2,3,4], x=2
Output:
3

Input:
arr=[10,20,20,30,30,30], x=30
Output:
3

Input:
arr=[5,5,5,5], x=9
Output:
0

Code:

```
public class CountOccurrences {
    static int firstOccurrence(int[] arr, int x) {
        int low = 0, high = arr.length - 1;
        int result = -1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == x) {
                result = mid;
                high = mid - 1;
            } else if (arr[mid] < x) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return result;
    }
    static int lastOccurrence(int[] arr, int x) {
        int low = 0, high = arr.length - 1;
        int result = -1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == x) {
                result = mid;
                low = mid + 1;
            } else if (arr[mid] < x) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return result;
    }
    static int countOccurrences(int[] arr, int x) {
        int first = firstOccurrence(arr, x);
        if (first == -1) {
            return 0;
        }
        int last = lastOccurrence(arr, x);
        return last - first + 1;
```

```
    }
    public static void main(String[] args) {
        int[] arr1 = {1, 1, 2, 2, 2, 3, 4};
        System.out.println(countOccurrences(arr1, 2));
        int[] arr2 = {10, 20, 20, 30, 30, 30};
        System.out.println(countOccurrences(arr2, 30));
        int[] arr3 = {5, 5, 5, 5};
        System.out.println(countOccurrences(arr3, 9));
    }
}
```

Output

```
3
3
0
```

19. Find the Equilibrium Index

An equilibrium index i satisfies: sum of arr[0..i-1] == sum of arr[i+1..n-1]. Return the first such index. Return -1 if none exists.

Note: Compute total sum first, then iterate updating left sum. Equilibrium when leftSum == totalSum - leftSum - arr[i].

Constraints:
1 <= n <= 10^5 | -10^4 <= arr[i] <= 10^4

Input
arr=[-7,1,5,2,-4,3,0]
Output
3

Input
arr=[1,2,3]
Output
-1

Input
arr=[0]
Output
0

Code:

```
import java.util.*;

public class EquilibriumIndex {

    public static int findEquilibriumIndex(int[] arr) {
        int totalSum = 0;
        for (int num : arr) {
            totalSum += num;
        }
        int leftSum = 0;
        for (int i = 0; i < arr.length; i++) {
            int rightSum = totalSum - leftSum - arr[i];
            if (leftSum == rightSum) {
                return i;
            }
            leftSum += arr[i];
        }

        return -1;
    }
    public static void main(String[] args) {
        int[] arr1 = {-7, 1, 5, 2, -4, 3, 0};
        System.out.println(findEquilibriumIndex(arr1));
        int[] arr2 = {1, 2, 3};
        System.out.println(findEquilibriumIndex(arr2));
        int[] arr3 = {0};
        System.out.println(findEquilibriumIndex(arr3));
    }
}
```

Output

```
3
-1
0
```

20. Count Pairs with Given Difference

Given an unsorted array arr[] and an integer diff, count the number of pairs (i, j) where i < j and arr[j] - arr[i] == diff.

Note: Sort the array, then use binary search to find arr[i] + diff for each element.

Constraints:
1 <= n <= 10^4 | 0 <= arr[i] <= 10^5 | 0 <= diff <= 10^5

Input
arr=[1,5,3,4,2], diff=3
Output
2 (pairs: (1,4) and (2,5))

Input
arr=[8,12,16,4,0,20], diff=4
Output
5

Input
arr=[1,1,1,1], diff=0
Output
6

Code:

```
import java.util.Arrays;

public class CountPairsWithGivenDifference {
    static boolean binarySearch(int[] arr, int target, int start) {
        int left = start;
        int right = arr.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (arr[mid] == target)
                return true;
            else if (arr[mid] < target)
                left = mid + 1;
            else
                right = mid - 1;
        }
        return false;
    }
    static int countPairs(int[] arr, int diff) {
        Arrays.sort(arr);
        int count = 0;
        if (diff == 0) {
            int i = 0;
            while (i < arr.length) {
                int freq = 1;
                while (i + 1 < arr.length && arr[i] == arr[i + 1]) {
                    freq++;
                    i++;
                }
                count += (freq * (freq - 1)) / 2;
                i++;
            }
            return count;
        }
        for (int i = 0; i < arr.length; i++) {
            if (binarySearch(arr, arr[i] + diff, i + 1)) {
                count++;
            }
        }
        return count;
    }
```

```
    public static void main(String[] args) {
        int[] arr1 = {1, 5, 3, 4, 2};
        int diff1 = 3;
        System.out.println(countPairs(arr1, diff1));
        int[] arr2 = {8, 12, 16, 4, 0, 20};
        int diff2 = 4;
        System.out.println(countPairs(arr2, diff2));
        int[] arr3 = {1, 1, 1, 1};
        int diff3 = 0;
        System.out.println(countPairs(arr3, diff3));
    }
}
```

Output

```
2
5
6
```